



ROYAL UNIVERSITY OF PHNOM PENH

Faculty of Engineering

Information Technology Engineering

ANALYSIS REPORT

CoolApp Mobile Application Using Ghidra

Course: *Project Practicum*

Instructor: *Mr. Kor Sokchea*

Student Name: *Keo Sopharanith - កែវ សុផារ៉ានីត*

Date: 1 April 2026

Phnom Penh, Cambodia

Keo Sopharanith - កែវ សុផារ៉ានីត.

Reverse Engineering Report: CoolApp Mobile Application Using Ghidra

Abstract

This report presents a static analysis of the *CoolApp* mobile application using the Ghidra framework. The objective of this analysis is to examine the internal structure, functional components, and execution logic of the application without executing it.

The study focuses on analyzing program behavior through control flow, function-level inspection, and data flow tracking. The findings demonstrate how static analysis techniques can effectively reveal internal architecture, execution patterns, and system-level behavior of the application.

Introduction

Modern mobile applications are composed of complex architectures with multiple interconnected modules. Understanding their internal behavior is essential for security analysis, debugging, and system evaluation.

In this study, *CoolApp* is analyzed using static analysis techniques. The APK file is extracted and examined using Ghidra to understand its internal structure and execution logic.

The objective of this report is to apply a structured methodology to analyze and interpret the behavior of a compiled application.

Theoretical Background

The analysis is based on key program analysis concepts:

- **Control Flow Analysis** Examines execution paths and branching logic within the program.
- **Data Flow Analysis** Tracks how data moves and is transformed throughout the application.
- **Abstract Interpretation** Estimates program behavior without actual execution.
- **Soundness vs. Precision** Balances completeness and accuracy in analysis results.

These concepts form the foundation for understanding program behavior during static analysis.

Tools Used

1. Ghidra (Primary Tool)

Ghidra is used due to its advanced reverse engineering capabilities:

- Disassembly and decompilation
- Control Flow Graph (CFG) visualization
- Function identification
- Program structure analysis

2. Supporting Tools

APK extraction tools for retrieving compiled code, resources, and libraries

Methodology

The analysis follows a structured workflow:

1. APK Extraction

- Extracted application package from Android device

- Retrieved compiled code and resources
2. **Static Analysis in Ghidra**
 - Imported APK into Ghidra
 - Executed auto-analysis
 3. **Program Analysis**
 - Examined decompiled code
 - Analyzed control flow graphs
 - Identified functions and execution logic

Analysis Workflow

- **Input:** APK File
- **Processing:**
 - Extraction
 - Static analysis
 - Code examination
- **Output:**
 - Program structure
 - Execution logic
 - System behavior

Analysis Results (Enhanced Technical Detail)

1. Control Flow Graph (CFG) Analysis

Control Flow Graphs (CFGs) were generated in Ghidra to analyze execution paths.

- Nodes represent basic blocks
- Edges represent control transitions

Findings:

- Conditional branching (if-else structures)
- Loop constructs for repeated operations
- Function call relationships

CFG analysis shows how the application controls execution flow and decision-making processes.

2. Function-Level Analysis (Based on Decompiled Code)

Entry Function (Main Execution Flow)

```
void entry(void)
```

This function represents the **program entry point**, responsible for initialization, command processing, and system-level operations.

❖ System Initialization

```
SetErrorMode(0x8001);  
DVar2 = GetVersion();  
InitCommonControls();  
OleInitialize((LPVOID)0x0);
```

Analysis:

- Initializes system environment
- Prepares GUI and COM components
- Ensures compatibility with OS

❖ Command-Line Processing

```
pWVar5 = GetCommandLineW();
```

Analysis:

- Retrieves execution arguments
- Supports flags such as /S (silent execution)
- Indicates configurable runtime behavior

❖ File and Temporary Directory Handling

```
GetTempPathW(...);  
DeleteFileW(...);  
CopyFileW(...);
```

Analysis:

- Creates temporary directories
- Deletes and copies files dynamically
- Suggests file extraction behavior

❖ Loop-Based File Operations

```
do {  
    DeleteFileW(...);  
    CopyFileW(...);  
} while (iVar4 != 0);
```

Analysis:

- Repeated file operations
- Ensures file creation or replacement
- CFG identifies this as a loop structure

❖ Privilege Escalation

```
OpenProcessToken(...);  
AdjustTokenPrivileges(...);
```

Analysis:

- Requests system-level privileges
- Enables advanced operations (e.g., shutdown)

❖ System Execution Control

```
ExitWindowsEx(...);  
ExitProcess(...);
```

Analysis:

- Controls system shutdown or restart
- Terminates program execution

❖ Unknown Internal Functions

```
FUN_0040335e();  
FUN_00402edd(...);
```

Analysis:

- Represent unidentified internal logic
- Likely contain core functionality
- Require deeper reverse engineering

Key Insight

The analyzed function does not directly represent application features like login or messaging. Instead, it demonstrates:

- System initialization
- File extraction behavior
- Command-line control
- Privilege management

This strongly suggests the application behaves like:

An installer or self-extracting executable rather than a simple mobile app module

3. Data Flow Analysis

Data flow tracking reveals:

- Input data flows through multiple internal buffers
- File data is written and copied across directories
- Parameters are passed between internal functions

Implications:

- Identifies how data is handled internally
- Helps detect potential security risks

4. Program Logic Patterns

Observed logic structures include:

- Conditional statements → decision-making
- Loops → repeated file operations
- Function calls → modular execution

These patterns indicate a structured execution model with system-level control.

Discussion

The analysis reveals that the examined code primarily handles system-level operations rather than high-level application features.

Control flow and function-level analysis indicate that the program performs initialization, file handling, and privilege management. This suggests behavior consistent with installer or packaged executable systems.

Static analysis proved effective in uncovering these internal behaviors without requiring program execution.

Conclusion

This report presented a static analysis of the CoolApp application using Ghidra. The study successfully identified execution flow, system-level behavior, and internal function structure.

The results demonstrate that static analysis is a powerful method for understanding compiled software, and Ghidra provides an effective platform for such analysis.

References

- Nielson, F., Nielson, H. R., & Hankin, C. (2005). *Principles of Program Analysis*. Springer.
- Ghidra Documentation, National Security Agency